

MINI PROJECT

Exploratory Data Analysis – Fastfood & Beverages
Pada Balaji Fastfood

Table of Contents

Tentang Saya

Project yang Dikerjakan Sebelumnya

Main Project:

- **Balaji Fastfood**
- **Problem Statement**
- **Data Understanding**
- **Data Preparation**
- **Visualization Output**

Conclusions & Recommendation

Personal Reflection



TENTANG SAYA

Moch Fadlilah Rahmadi adalah Sarjana Teknik Industri lulusan Universitas Bina Nusantara dengan pengalaman lebih dari 3 tahun di bidang pengelasan pipa kabel, manajemen produksi, optimasi proses, dan analisis ergonomi di industri manufaktur. Terampil dalam mengimplementasikan peningkatan alur kerja dan inisiatif keselamatan untuk meningkatkan efisiensi operasional dan memastikan kepatuhan terhadap standar tempat kerja. Mahir dalam menerapkan alat analisis dan metode penilaian ergonomi seperti OWAS dan Nordic Body Map untuk mempromosikan lingkungan kerja yang lebih aman dan efisien. Dikenal karena kemampuan adaptasi yang kuat, kerja tim, dan pola pikir perbaikan berkelanjutan di lingkungan multikultural.

PENGALAMAN KERJA

Prysmian Group

Maret 2022–January 2026

Wijaya Karya
Industri & Konstruksi

Maret 2021–Mei 2021

PENDIDIKAN

Data Analyst Bootcamp

Dibimbing.id

2026–Sekarang

Teknik Industri

Universitas Bina Nusantara

2023–2025

Teknik Mesin

Politeknik Negeri Jakarta

2018–2021



PROJECT YANG DIKERJAKAN SEBELUMNYA

01 PROJECT OVEN SANDCORE

Project pembuatan oven sandcore untuk mengurangi product reject saat melakukan proses casting di PT.Wijaya Karya Industri & Konstruksi. Project ini berhasil mengurangi cacat produksi akibat oksidasi sebesar 10%.

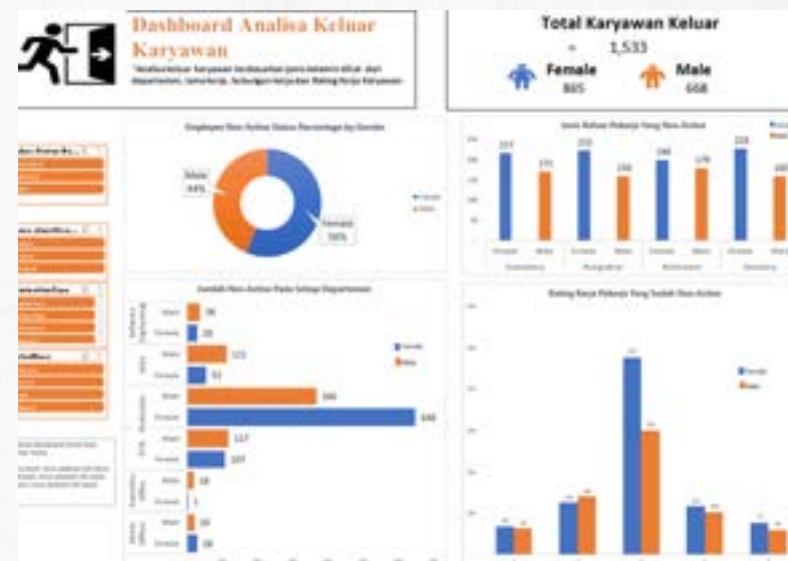


03 PROJECT DATA PREPROCESSING

Project ini merupakan project yang mana diberikannya data set perusahaan fastfood sebanyak 1000 baris dan 10 kolom yang masih terdapat missing dan type data yang belum sesuai. Hasil dari project ini data yang di preprocessing sudah siap untuk analisa

02 PROJECT DASHBOARD ANALISA KELUAR KARYAWAN

Menganalisa keluar karyawan berdasarkan jenis-kelamin dilihat dari departemen, lama kerja, hubungan kerja dan Rating Kerja Karyawan. Lalu membuatkan dashboard untuk melihat secara keseluruhannya mengenai permasalahan tersebut.



```
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('float')
final_data = final_data.select_dtype('
```


MAIN PROJECT

BALAJI FASTFOOD

Balaji Fastfood merupakan suatu usaha yang bergerak pada bidang F&B (food and beverages) yang menyajikan makanan dan minuman khas dari India. Usaha ini sudah berdiri sejak tahun 2008 yang mana cabang utamanya terletak di bhayandar,Mumbai. Usaha ini juga sudah memiliki beberapa cabang di India.

Menu yang dihidangkan oleh Balaji Fastfood:

Aalopuri



Sandwich



Vadapav



Panipuri



Frankie



Cold Coffee



Sugarcane Juice



PROBLEM STATEMENT

Balaji Fast Food adalah usaha kuliner yang melayani berbagai jenis makanan dengan metode pembayaran Cash dan Online. Meskipun memiliki data transaksi yang cukup lengkap, Akan tetapi **manajemen belum memahami pola penjualan secara mendalam — produk mana yang paling menguntungkan, waktu puncak penjualan, dan perilaku pelanggan berdasarkan metode pembayaran.**

Menurut jurnal dari International Journal of Gastronomy and Food Science yang berjudul “Menu fatigue: Exploring an obscure concept with problem-centred expert interviews” **hampir 80% dari usaha fastfood yang ada di dunia kebingungan dalam menentukan produk apa yang sebaiknya mereka jual hingga konsep penjualan apa yang mereka perlukan.**

(<https://doi.org/10.1016/j.ijgfs.2024.101047>)

Pertanyaan Bisnis :

1. Produk apa yang paling laris dan paling menghasilkan revenue?
2. Kapan waktu penjualan tertinggi (time of sale) berdasarkan transaction amountnya?
3. Apakah metode pembayaran (Cash vs Online) mempengaruhi transaction amount?
4. Faktor apa yang paling berkorelasi dengan tingginya transaction_amount?**

DATA UNDERSTANDING

DATA SET

1. Data set yang saya gunakan pada project kali ini diambil dari google drive yang diberikan oleh mentor, dan **format file yang diberikan dalam bentuk csv**. Berikut adalah link untuk mengakses data set yang saya miliki:

https://drive.google.com/file/d/13Q-VpNJ-PpP0lqKJdXSpoxDPe6qvUu3/view?usp=drive_link

```
data_ff.shape
```

```
(1000, 10)
```

```
data_ff.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 1000 entries, 0 to 999  
Data columns (total 10 columns):
```

2. Data yang akan diolah dan digunakan memiliki **jumlah kolom sebanyak 10 dan panjang baris sebanyak 1000**.

DATA SET

No.	Kolom	Deskripsi
1	order_id	Identifikasi unik untuk setiap pesanan
2	date	Tanggal transaksi
3	item_name	Nama makanan
4	item_type	Kategori item (Makanan Cepat Saji atau Minuman)
5	item_price	Harga item untuk 1 porsi/satuan
6	Quantity	Jumlah item yang dipesan oleh pelanggan
7	transaction_amount	Total jumlah uang yang dibayarkan oleh pelanggan
8	transaction_type	Metode pembayaran (tunai, online, lainnya)
9	received_by	Jenis kelamin petugas yang menangani transaksi
10	time_of_sale	Waktu yang berbeda dalam sehari (Pagi, Siang, Sore, Malam, Tengah Malam)

#	Column	Non-Null Count	Dtype
0	order_id	1000 non-null	int64
1	date	1000 non-null	object
2	item_name	1000 non-null	object
3	item_type	1000 non-null	object
4	item_price	1000 non-null	int64
5	quantity	1000 non-null	int64
6	transaction_amount	1000 non-null	int64
7	transaction_type	893 non-null	object
8	received_by	1000 non-null	object
9	time_of_sale	1000 non-null	object

Data set memiliki kolom deskripsi yang beragam akan tetapi type data pada setiap kolomnya ada yang memiliki type yang sama.

IMPORT DATA

Langkah-langkah melakukan import data:

- 1. Menyiapkan tools yang akan digunakan untuk mengolah dan mengimpor data seperti pandas, numpy dsb.**
- 2. Memuat atau memanggil data set dari file csv.**
- 3. Pada project kali ini, penulis memanggil file dengan pandas untuk ditampilkan di notebook.**

Output dari import Data:

Dataset sukses untuk dimuat pada data frame yang akan digunakan.

Import Data

```
[2]: #step 1, insert data balaji ff sales
data_ff = pd.read_csv(r'C:\Users\Fadlilah\Downloads\Balaji Fast Food Sales (tb & fadli).csv') #memanggil data ff
data_ff #hal yang pertama kali dilakukan ialah mengimpor data yang akan digunakan sebagai langkah awal analisa.
# data yang saya import merupakan data yang sudah saya download dan saya simpan di lokal dan saya memanggil data tersebut dengan pandas dengan format pa
```

[illegible]

DATA EXPLORATION

Tujuan:

Memahami struktur dan isi dari dataset sebelum melakukan cleaning data dan juga modelling

#action 1

```
#data exploration 2, info() saya gunakan untuk melihat data type  
data_ff.info() #untuk melihat info nilai dari atribut yang ada
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 1000 entries, 0 to 999  
Data columns (total 10 columns):  
#   Column                Non-Null Count  Dtype    
---  -  
0   order_id              1000 non-null   int64    
1   date                  1000 non-null   object   
2   item_name             1000 non-null   object   
3   item_type             1000 non-null   object   
4   item_price            1000 non-null   int64    
5   quantity              1000 non-null   int64    
6   transaction_amount     1000 non-null   int64    
7   transaction_type       893 non-null    object   
8   received_by           1000 non-null   object   
9   time_of_sale           1000 non-null   object   
dtypes: int64(4), object(6)  
memory usage: 78.3+ KB
```

#action 2

```
#data exploration 5, fungsi isnull() dan sum() saya gunakan untuk m  
data_ff.isnull().sum()
```

```
order_id      0  
date          0  
item_name     0  
item_type     0  
item_price    0  
quantity      0  
transaction_amount  0  
transaction_type  107  
received_by   0  
time_of_sale  0  
dtype: int64
```

#action 3

```
data_ff.duplicated().sum()
```

```
np.int64(0)
```

outputnya”

Pada tahap data exploration ini, di action 1 peneliti melakukan pengecekan type data terhadap semua kolom yang ada. Lalu dilanjutkan dengan action 2 yaitu pengecekan missing value pada setiap kolomnya, dan ditemukan bahwa ada missing value berjumlah 107 pada data di kolom transaction amount. Yang terakhir action 3 pengecekan duplicated pada data, dan hasilnya tidak ditemukan adanya duplikat

DATA QUALITY ISSUE

Missing Value

- Terdapat missing value pada kolom transaction_type

Mendeteksi Duplikat

- Tidak terdapat duplikat pada data yang ada

Mengidentifikasi outliers dan nilai yang tidak normal

- Terdapat outlier pada kolom transaction_amount
- Outlier pada transaction amount terdeteksi pada transaction >500

DATA PREPARATION

DATA CLEANING

Penanganan Missing Value

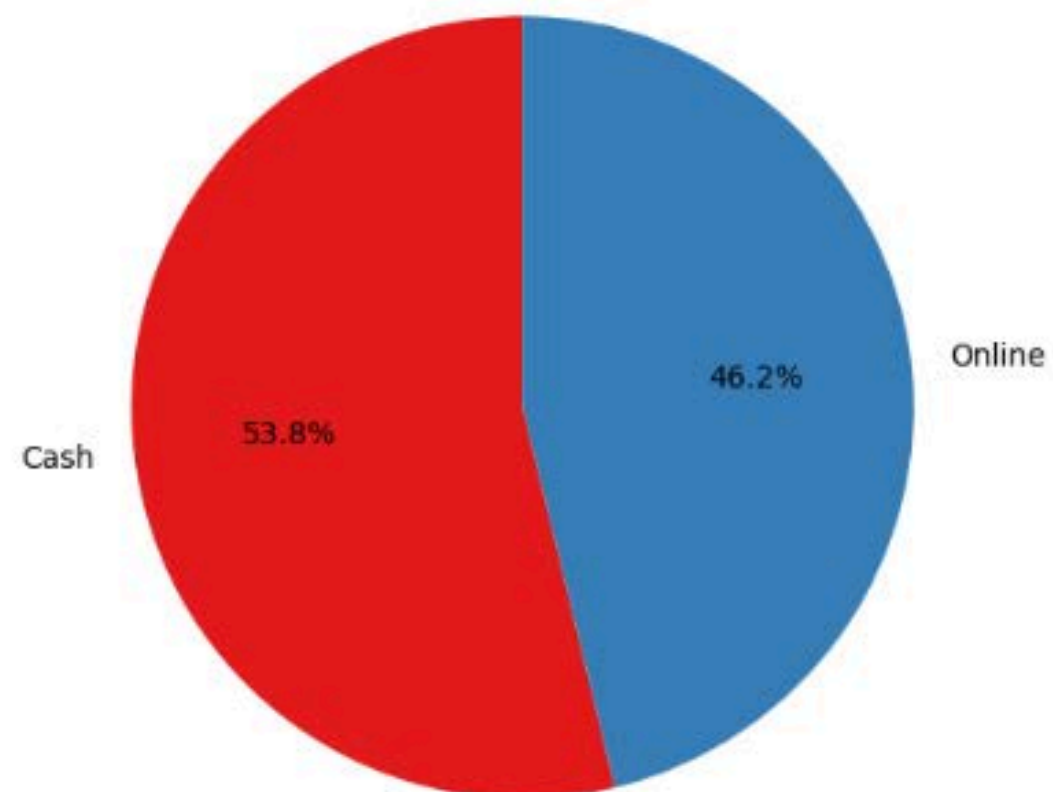
1. Kolom transaction_type

- Terdapat 107 data kosong pada kolom transaction_amount.
- Mengisi missing values menggunakan random seed dari distribusi data asli sehingga nantinya proporsi antara opsi transaction amount-nya tidak terlalu jauh. Selain itu juga tidak mengubah proporsi data antara cash dan online.

Transaction Type Sebelum Cleaning



Transaction Type Cleaning



```
#data clean 5d, memastikan k  
dataclean.isnull().sum()
```

```
order_id      0  
date          0  
item_name     0  
item_type     0  
item_price    0  
quantity      0  
transaction_amount  0  
transaction_type  0  
received_by   0  
time_of_sale  0  
dtype: int64
```

```
#data cleaning 5b, mengisi missing value pada transaction_type  
#memanggil fungsi random seed terlebih dahulu, karena saya akan mengisi missing value d  
np.random.seed(42) #42 pada dalam tutup kurung tidak memiliki makna itu hanya kebiasa  
  
# menghitung total data yang tidak missing terlebih dahulu  
total = 476 + 417 # 476 merupakan jumlah yang cash dan 417 merupakan jumlah yang online  
  
#membuat inputan yang akan dimasukkan ke missing value  
isi = dataclean['transaction_type'].isin([])|dataclean['transaction_type'].isnull()  
  
#mengisi missing value dengan inputan isi yang sudah kita buat  
dataclean.loc[isi,'transaction_type']=np.random.choice(  
    ['Cash','Online'],  
    size=isi.sum(),  
    p=[476/total, 417/total]  
)  
dataclean['transaction_type']
```

DATA CLEANING

Mendeteksi Ulang Duplikasi

- Tidak Terdapat Duplikat Pada Dataset

```
#data clean 5d, memastikan k  
dataclean.isnull().sum()
```

```
order_id      0  
date          0  
item_name     0  
item_type     0  
item_price    0  
quantity      0  
transaction_amount  0  
transaction_type  0  
received_by   0  
time_of_sale  0  
dtype: int64
```

Mendeteksi Ulang inkonsistensi

- Tidak Terdapat Inkonsistensi Pada Dataset

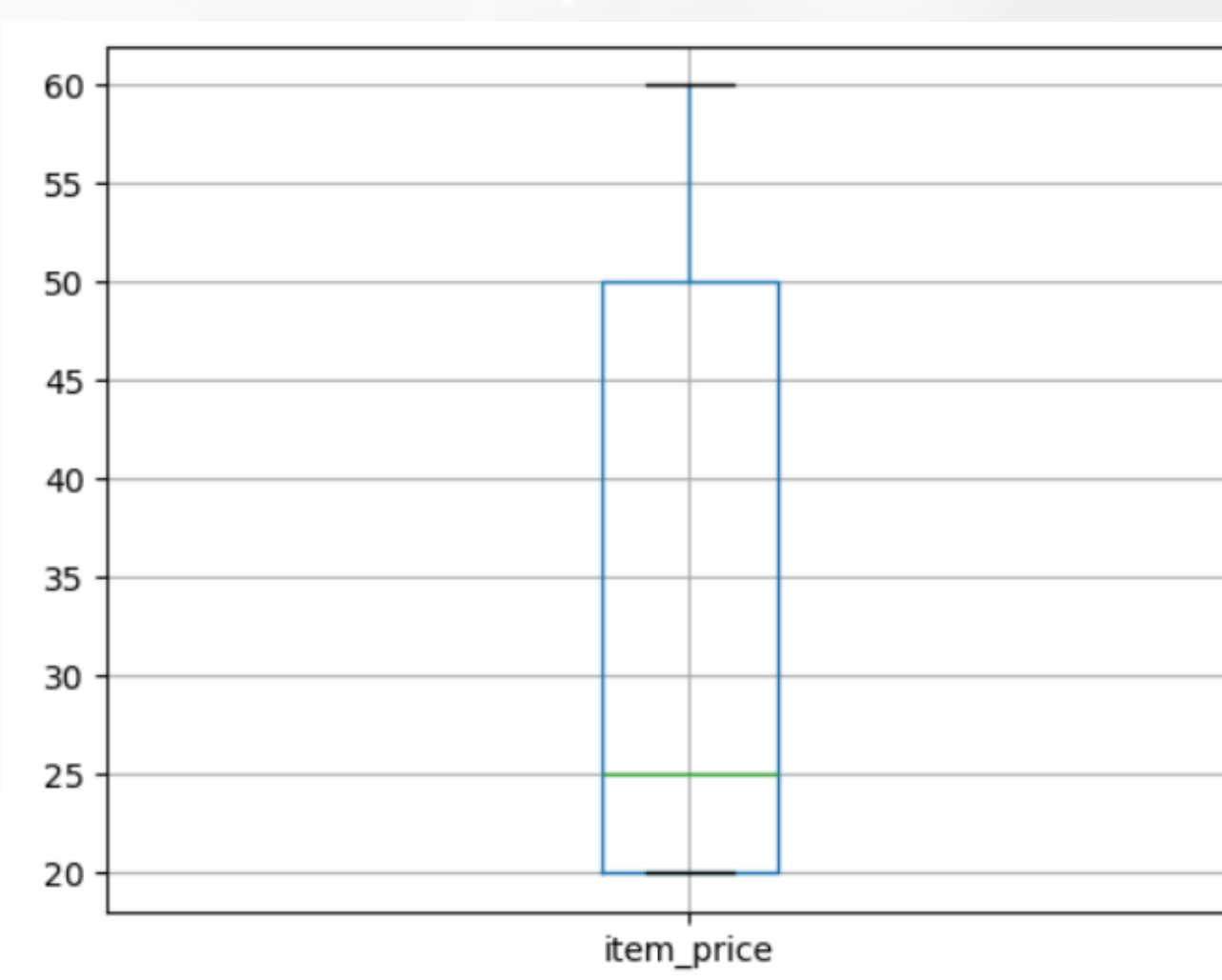
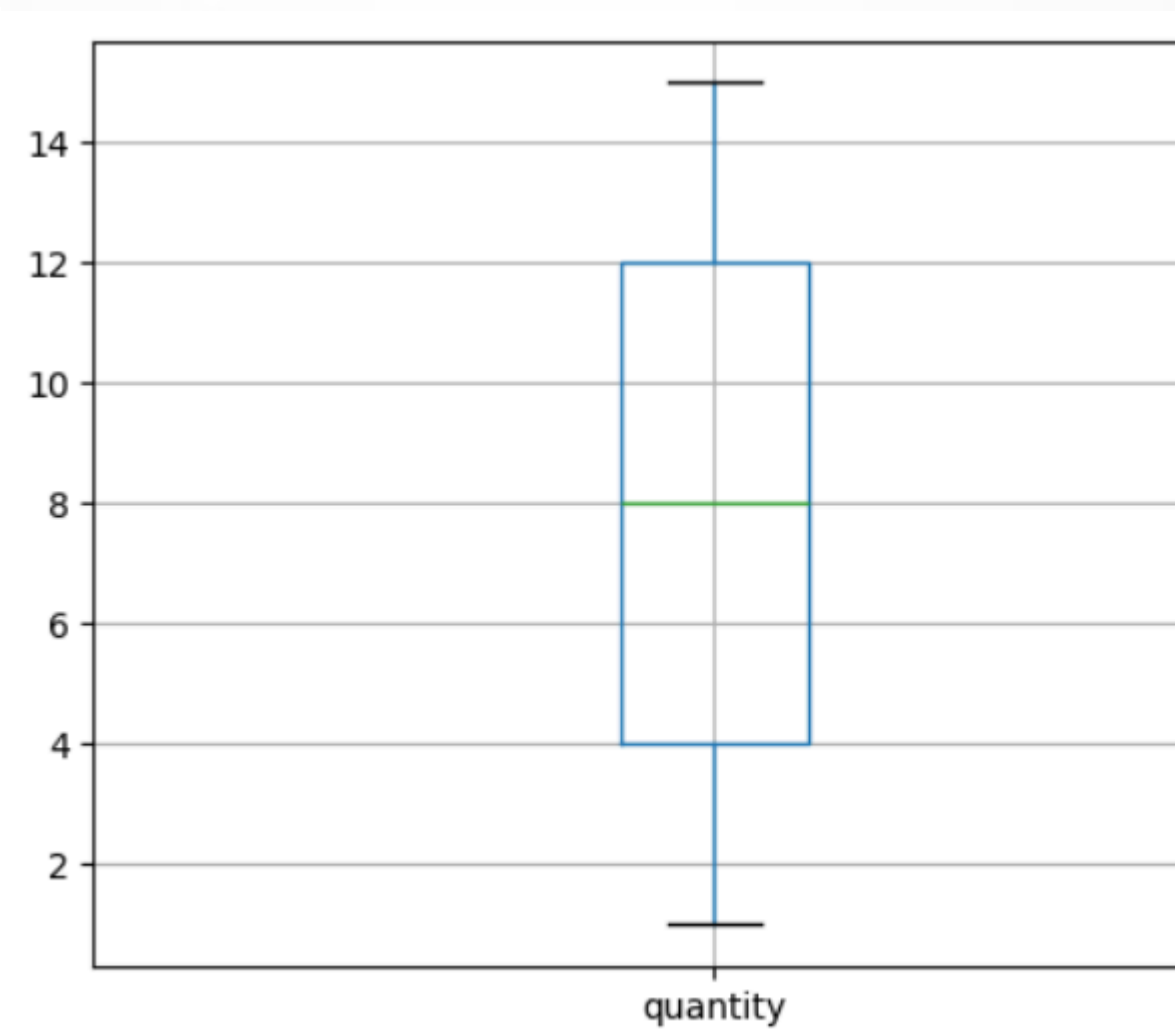
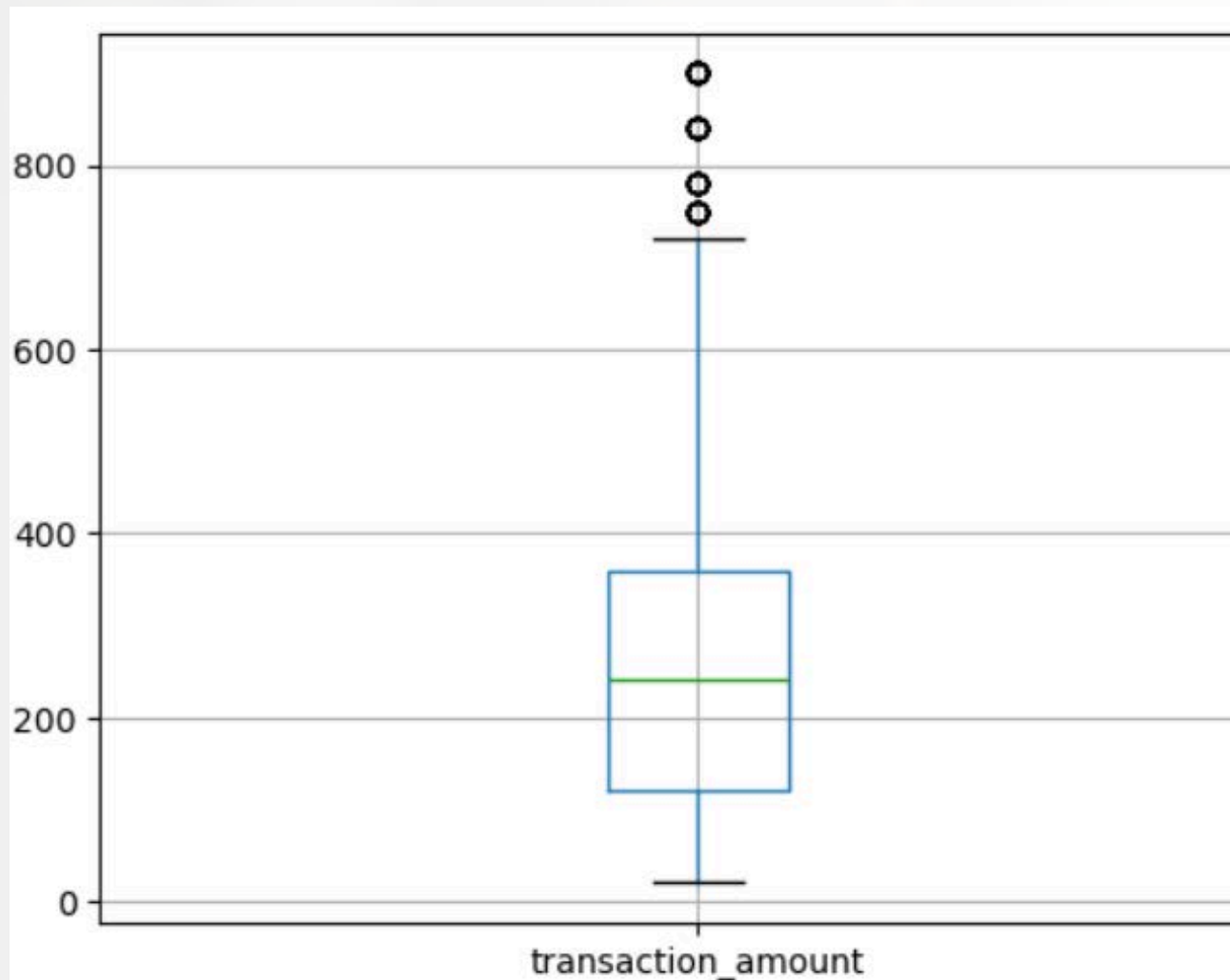
```
#data cleaning 1, saya menggunakan unique() untuk menampilkan atribut apa  
for col in data_ff.columns:  
    print(f"\nkolom:{col}")  
    print(data_ff[col].unique())
```

```
kolom:order_id  
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14  
15 16 17 18 19 20 21 22 23 24 25 26 27 28  
29 30 31 32 33 34 35 36 37 38 39 40 41 42  
43 44 45 46 47 48 49 50 51 52 53 54 55 56  
57 58 59 60 61 62 63 64 65 66 67 68 69 70  
71 72 73 74 75 76 77 78 79 80 81 82 83 84  
85 86 87 88 89 90 91 92 93 94 95 96 97 98  
99 100 101 102 103 104 105 106 107 108 109 110 111 112  
113 114 115 116 117 118 119 120 121 122 123 124 125 126  
127 128 129 130 131 132 133 134 135 136 137 138 139 140  
141 142 143 144 145 146 147 148 149 150 151 152 153 154  
155 156 157 158 159 160 161 162 163 164 165 166 167 168  
169 170 171 172 173 174 175 176 177 178 179 180 181 182  
183 184 185 186 187 188 189 190 191 192 193 194 195 196  
197 198 199 200 201 202 203 204 205 206 207 208 209 210  
211 212 213 214 215 216 217 218 219 220 221 222 223 224  
225 226 227 228 229 230 231 232 233 234 235 236 237 238  
239 240 241 242 243 244 245 246 247 248 249 250 251 252]
```

DATA CLEANING

Deteksi Outlier menggunakan metode visual dan IQR

Dalam dataset yang dimiliki, identifikasi dilakukan menggunakan metode Interquartile Range (IQR) untuk melihat persebaran data pada kolom-kolom yang numerik.



Temuan:

Dari pengecekan visual, terdapat outlier pada kolom transaction_amount yang dimiliki oleh dataset

DATA CLEANING

Deteksi Outlier menggunakan metode visual dan IQR

Dalam dataset yang dimiliki, identifikasi dilakukan menggunakan metode Interquartile Range (IQR) untuk melihat persebaran data pada kolom-kolom yang numerik.

```
#data cleaning 6b, mengidentifikasi dan menangani outlier dengan metode iqr
#transaction_amount outlier
#deteksi outlier transac menggunakan iqr
Q1 = dataclean["transaction_amount"].quantile(0.25)
Q3 = dataclean["transaction_amount"].quantile(0.75)
IQR = Q3-Q1

lower = Q1 - 1.5*IQR
upper = Q3 + 1.5*IQR

outlier_iqr = dataclean[(dataclean["transaction_amount"]<lower)|
(dataclean["transaction_amount"]>upper)]
outlier_iqr
```

```
#data cleaning 6c, menangani outlier dalam artian pengambilan keputusan lanjutan
jumlah_outlier = len(outlier_iqr)
print(f"jumlah outlier: {jumlah_outlier}")

#kalkulasi outlier dianggap aman
kalkulasi_outlier_transac = (jumlah_outlier/1000)*100
if (jumlah_outlier/1000)*100<5:
    print(f"persentase outlier_transaction:{kalkulasi_outlier_transac}%", "Aman")
else:
    print(f"persentase outlier_transaction:{kalkulasi_outlier_transac}%", "Tdk-Aman")
```

```
jumlah outlier: 44
persentase outlier_transaction:4.3999999999999995% Aman
```

Temuan:

Dari pengecekan menggunakan perhitungan IQR, **terdapat outlier pada kolom transaction_amount sebanyak 44 data.** Lalu peneliti melakukan kalkulasi apakah data tersebut aman untuk tidak dihapus. Setelah dilakukan kalkulasi ternyata **data outlier tersebut aman untuk tidak dihapus.** Kenapa aman, karena outlier pada transaction_amount merupakan hal wajar pada pembelian, bisa jadi ada pelanggan yang melakukan pembelian dalam jumlah besar dengan item_price yang mahal.

DATA MANIPULATION

Uji Normalitas

- Uji Shapiro-Wilk digunakan untuk menguji apakah distribusi normal.
- Uji ini merupakan standar dalam analisis statistika untuk menentukan apakah data dapat diolah dengan metode Parametrik (seperti ANOVA) atau harus menggunakan metode Non-Parametrik (seperti Kruskal-Wallis).

Temuan

- Karena nilai p-value < 0.05 , Hal ini membuktikan bahwa distribusi data kedua kolom menyimpang secara nyata dari distribusi normal
- Perlu dilakukan pengujian untuk transformasi data/normalisasi data

```
from scipy.stats import shapiro

for seg in dataclean['item_name'].unique():
    stat, p = shapiro(dataclean[dataclean['item_name']
    print(seg, "p-value:", p)
```

```
Aalopuri p-value: 1.2064204909506756e-07
Vadapav p-value: 3.372792119551031e-06
Sugarcane juice p-value: 4.3849814013775645e-07
Panipuri p-value: 1.5741357344610355e-06
Frankie p-value: 0.00023235037033317727
Sandwich p-value: 1.681688319348265e-05
Cold coffee p-value: 1.273431287238342e-05
```

```
#data manipulation 3, melakukan pengecekan distribusi normal dan uji normalitas
#1. transaction_amount terhadap item_name
for seg in dataclean['item_name'].unique():
    stat, p = shapiro(dataclean[dataclean['item_name']==seg]['transaction_amount'])
    print(seg, "p-value:", p)
    if p > 0.05:
        print("Data terdistribusi normal")
    else:
        print("Data tidak terdistribusi normal")
```

```
Aalopuri p-value: 1.2064204909506756e-07
Data tidak terdistribusi normal
Vadapav p-value: 3.372792119551031e-06
Data tidak terdistribusi normal
Sugarcane juice p-value: 4.3849814013775645e-07
Data tidak terdistribusi normal
Panipuri p-value: 1.5741357344610355e-06
Data tidak terdistribusi normal
Frankie p-value: 0.00023235037033317727
Data tidak terdistribusi normal
Sandwich p-value: 1.681688319348265e-05
Data tidak terdistribusi normal
Cold coffee p-value: 1.273431287238342e-05
Data tidak terdistribusi normal
```

DATA MANIPULATION

Upaya Normalitas

Melakukan transformasi matematika pada data asli untuk menarik nilai-nilai ekstrem (outlier) agar distribusi data mendekati distribusi normal (lonceng), sehingga memenuhi asumsi pengujian statistik parametrik.

- Metode yang dipakai:
- Log: mengubah data asli menjadi nilai logaritma nya
- Square Root: setiap nilai dalam dataset digantikan dengan akar kuadrat dari nilai tersebut
- Boxcox: Metode otomatis yang mencari nilai pangkat / lambda
- paling optimal untuk menormalkan distribusi data.

Temuan

- Setelah dilakukan transformasi berkali-kali pada setiap data yang memiliki type numeriknya, tidak ada data yang berhasil mencapai distribusi normal.
- Setelah diuji 3 kali, peneliti melakukan uji non-parametrik kruskal. Dan salah satu contohnya pada quantity, dan hasilnya p_value lebih besar dari 0.05 yang menandakan tidak terdapat perbedaan distribusi quantity yang signifikan antar item_name.

```
#uji normalitas lagi menggunakan data boxcox_quantity
```

```
from scipy.stats import shapiro
```

```
for seg in dataclean['item_name'].unique():  
    stat, p = shapiro(dataclean[dataclean['item_name'] == seg]['boxcox_quantity'])  
    print(seg, "p-value:", p)
```

```
Aalopuri p-value: 2.1041739126765067e-07
```

```
Vadapav p-value: 2.147224108225326e-05
```

```
Sugarcane juice p-value: 2.1856342206872094e-07
```

```
Panipuri p-value: 8.470894456266035e-07
```

```
Frankie p-value: 0.0003000224910931083
```

```
Sandwich p-value: 8.218378974440886e-06
```

```
Cold coffee p-value: 4.210229618859041e-06
```

```
#7, ganti uji non-parametrik kruskal untuk mengecek data quantity yang tidak nor
```

```
groups = []
```

```
for g in dataclean['item_name'].unique():  
    groups.append(dataclean[dataclean['item_name'] == g]['quantity'])
```

```
h_stat, p_val = stats.kruskal(*groups)
```

```
print("kruskal-wallis test")
```

```
print("h-statistic:", h_stat)
```

```
print("p-value:", p_val)
```

```
alpha = 0.05
```

```
if p_val < alpha:  
    print("hasil signifikan: ada perbedaan distribusi antar item_name")
```

```
else:  
    print("hasil signifikan: tidak ada perbedaan distribusi antar item_name")
```

```
kruskal-wallis test
```

```
h-statistic: 5.510923121138764
```

```
p-value: 0.4801374813112068
```

```
hasil signifikan: tidak ada perbedaan distribusi antar item_name
```


DATA MANIPULATION

Mengubah Data Type

- Mengubah format tulisan yang semulanya menggunakan '/' menjadi '-' pada kolom date lalu mengubah type data yang semulanya bertipe data object menjadi format waktu.
- Mengubah semua type data object menjadi categorical.

```
#data manipulation 4, mengubah format penulisan date yang awalnya menggunakan / sebagai pemisah
dataclean['date'] = dataclean['date'].str.replace('/', '-')
dataclean['date'] = pd.to_datetime(dataclean['date'], format='%m-%d-%Y')
dataclean.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   order_id              1000 non-null  int64
1   date                  1000 non-null  datetime64[ns]
2   item_name             1000 non-null  object
3   item_type             1000 non-null  object
4   item_price            1000 non-null  int64
5   quantity              1000 non-null  int64
6   transaction_amount    1000 non-null  int64
7   transaction_type      1000 non-null  object
8   received_by           1000 non-null  object
9   time_of_sale          1000 non-null  object
10  log_quantity          1000 non-null  float64
11  sqrt_quantity         1000 non-null  float64
12  boxcox_quantity       1000 non-null  float64
13  boxcox_transaction_amount 1000 non-null  float64
dtypes: datetime64[ns](1), float64(4), int64(4), object(5)
memory usage: 109.5+ KB
```

```
#data manipulation 5, mengubah type data object menjadi category agar lebih efisien
cat_cols = ['transaction_type', 'item_name', 'item_type', 'received_by',

for col in cat_cols:
    dataclean[col] = dataclean[col].astype('category')

dataclean.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   order_id              1000 non-null  int64
1   date                  1000 non-null  datetime64[ns]
2   item_name             1000 non-null  category
3   item_type             1000 non-null  category
4   item_price            1000 non-null  int64
5   quantity              1000 non-null  int64
6   transaction_amount    1000 non-null  int64
7   transaction_type      1000 non-null  category
8   received_by           1000 non-null  category
9   time_of_sale          1000 non-null  category
10  log_quantity          1000 non-null  float64
11  sqrt_quantity         1000 non-null  float64
12  boxcox_quantity       1000 non-null  float64
13  boxcox_transaction_amount 1000 non-null  float64
dtypes: category(5), datetime64[ns](1), float64(4), int64(4)
memory usage: 76.2 KB
```

DATA MANIPULATION

Melakukan drop atau membuang kolom yang tidak terpakai

- meng-drop kolom, boxcox_quantity, boxcox_transaction_amount, log_quantity, dan sqrt quantity
- Alasan melakukan drop pada kolom tersebut ialah untuk karena data aslinya yang digunakan untuk analisa lanjutan.

```
#data manipulation 7, saya mengdrop kolom yang sekiranya tidak terpakai dan membuat copy untuk data cleaning final
dataclean_final = dataclean.copy()
dataclean_final = dataclean_final.drop(columns=[
    'log_quantity',
    'sqrt_quantity',
    'boxcox_quantity',
    'log_itemp',
    'sqrt_itemp',
    'boxcox_itemp',
    'log_transac',
    'sqrt_transac',
    'boxcox_transac'
])
```

	order_id	date	item_name	item_type	item_price	quantity	transaction_amount	transaction_type	received_by	time_of_sale
0	1	2022-07-03	Aalopuri	Fastfood	20	13	260	Cash	Mr.	Night
1	2	2022-08-23	Vadapav	Fastfood	20	15	300	Cash	Mr.	Afternoon
2	3	2022-11-20	Vadapav	Fastfood	20	1	20	Cash	Mr.	Afternoon
3	4	2023-02-03	Sugarcane juice	Beverages	25	6	150	Online	Mr.	Night
4	5	2022-10-02	Sugarcane juice	Beverages	25	8	200	Online	Mr.	Evening

DATA MANIPULATION

Melakukan groupby dan melakukan sort untuk melihat total quantity per item.

```
# Panel 1: Total Quantity per Item
qty_item = dataclean_final.groupby('item_name', observed=True)['quantity'].sum().sort_values(ascending=False)
```

```
item_name
Cold coffee      1361
Sugarcane juice  1278
Panipuri          1226
Frankie           1150
Sandwich          1097
Aalopuri          1044
Vadapav           1006
Name: quantity, dtype: int64
```

Hasil dari groupby di samping dapat terlihat bahwa cold coffee merupakan item dengan quantity pembelian terbanyak,. Sehingga kita dapat mengindikasikan bahwa cold coffee merupakan product favorite pengunjung (buyer).

Melakukan groupby untuk menghitung total transaction amount berdasarkan transaction type

```
rev_type = dataclean_final.groupby('transaction_type', observed=True)['transaction_amount'].sum()
```

Hasil dari groupby di samping dapat terlihat bahwa cash merupakan type transaction yang paling berpengaruh pada transaction amount.

```
transaction_type
Cash      151280
Online    123950
Name: transaction_amount, dtype: int64
```


DATA MANIPULATION

Melakukan groupby dan melakukan sort untuk melihat transaction amount per item.

```
item_revenue = dataclean_final.groupby('item_name', observed=True)['transaction_amount'].sum().sort_values(ascending=False).reset_index()
```

item_revenue

	item_name	transaction_amount
0	Sandwich	65820
1	Frankie	57500
2	Cold coffee	54440
3	Sugarcane juice	31950
4	Panipuri	24520
5	Aalopuri	20880
6	Vadapav	20120

Hasil dari groupby di samping dapat terlihat bahwa sandwich merupakan item dengan transaction amount terbesar. Sehingga kita dapat mengindikasikan bahwa sandwich merupakan product paling berpengaruh ke total transaction amount.

Menggunakan crosstab untuk melihat revenue tiap kategori produk berdasarkan metode pembayaran.

Hasil dari crosstab disamping nanti akan dapat melihat apakah customer cenderung menggunakan type pembayaran cash atau online untuk pembelian beverages maupun fastfood.

1. Cross Tabulation: Revenue berdasarkan Item Type & Transaction Type

Crosstab revenue (bukan frekuensi)

```
ct_revenue = pd.crosstab(
    dataclean_final['item_type'],
    dataclean_final['transaction_type'],
    values=dataclean_final['transaction_amount'],
    aggfunc='sum',
    margins=True,
    margins_name='Total'
).round(0)

print('=== Cross Tabulation: Revenue per Item Type x Transaction Type ===')
print(ct_revenue)
```

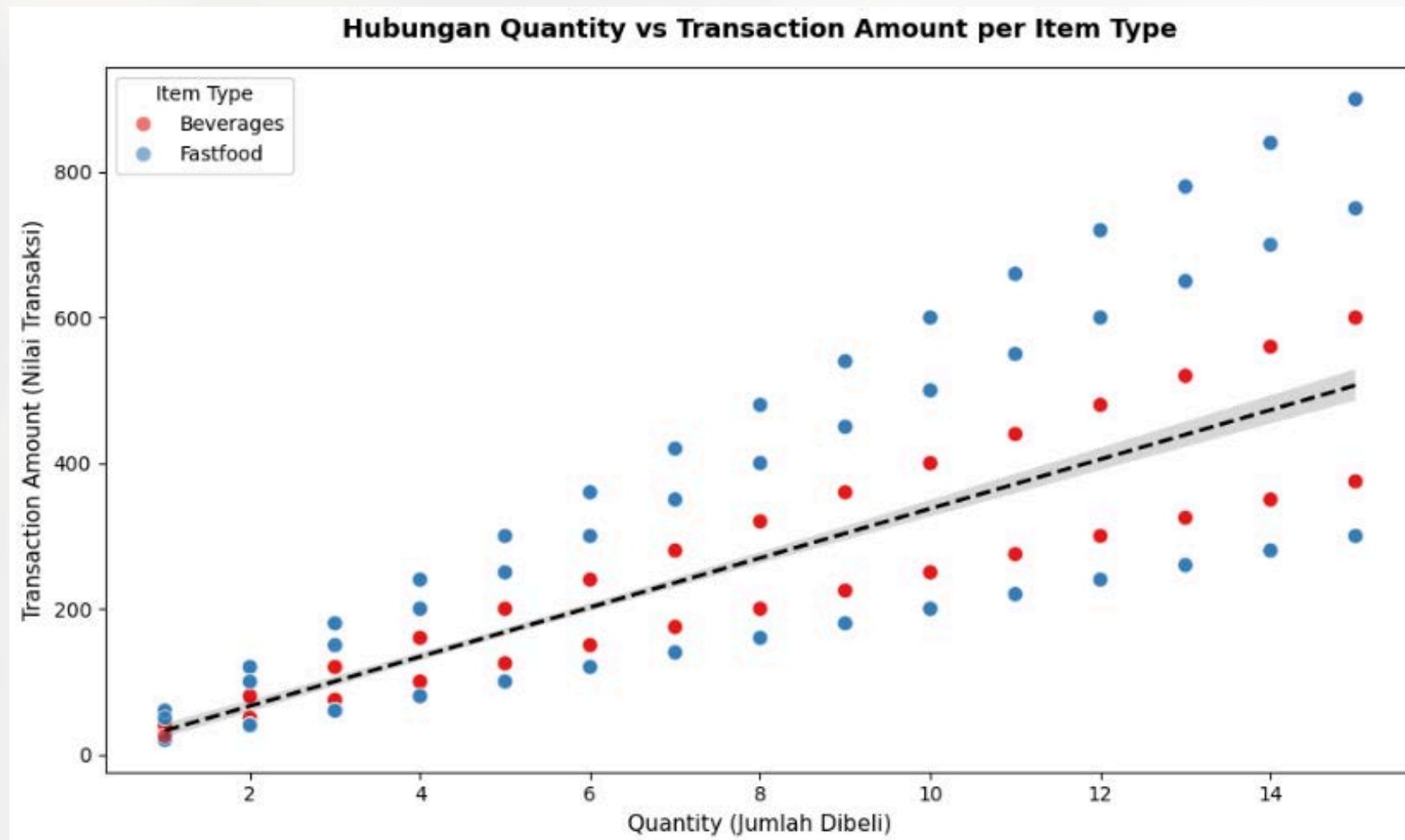
Crosstab persentase revenue (normalize per baris)

```
ct_revenue_pct = pd.crosstab(
    dataclean_final['item_type'],
    dataclean_final['transaction_type'],
    values=dataclean_final['transaction_amount'],
    aggfunc='sum',
    normalize='index'
).round(3) * 100

print('\n=== Cross Tabulation: Persentase Revenue per Item Type ===')
print(ct_revenue_pct)
```

OUTPUT VISUALIZATION

VISUALIZATION OUTPUT



Uji Hipotesis

- Menggunakan uji Spearman dikarenakan data tidak berdistribusi normal
- Untuk menganalisa pengaruh quantity terhadap transaction amount, digunakan hipotesa berikut:
 - H_1 = hasil signifikan: ada hubungan signifikan antara quantity dan transaction amount.
 - H_0 = hasil tidak signifikan: tidak ada hubungan signifikan antara quantity dan transaction amount

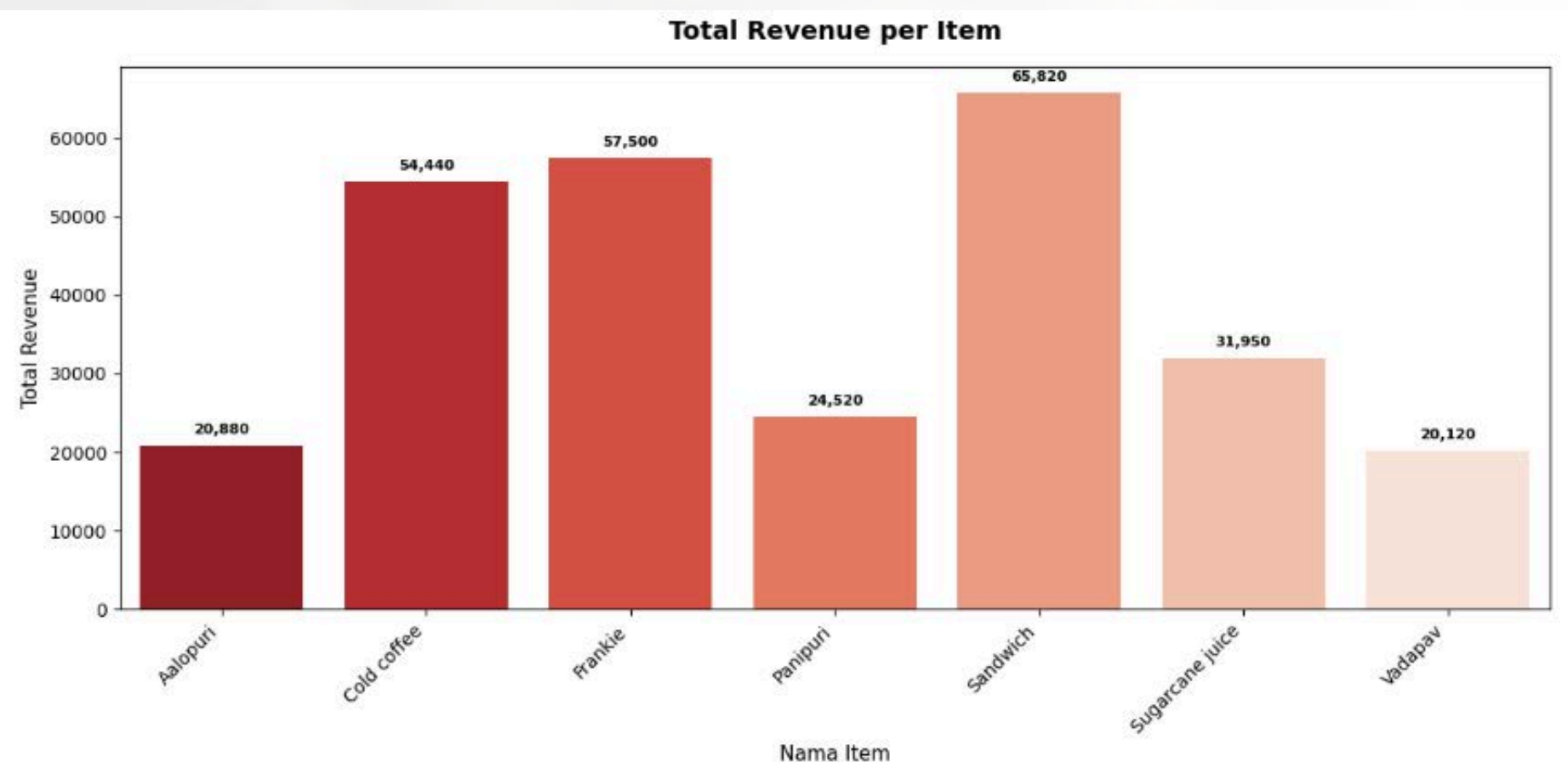
Hasil

```
Correlation: 0.8064271045085807
P-value: 6.085501337536874e-230
hasil signifikan: ada hubungan signifikan antara quantity dan transaction amount
```

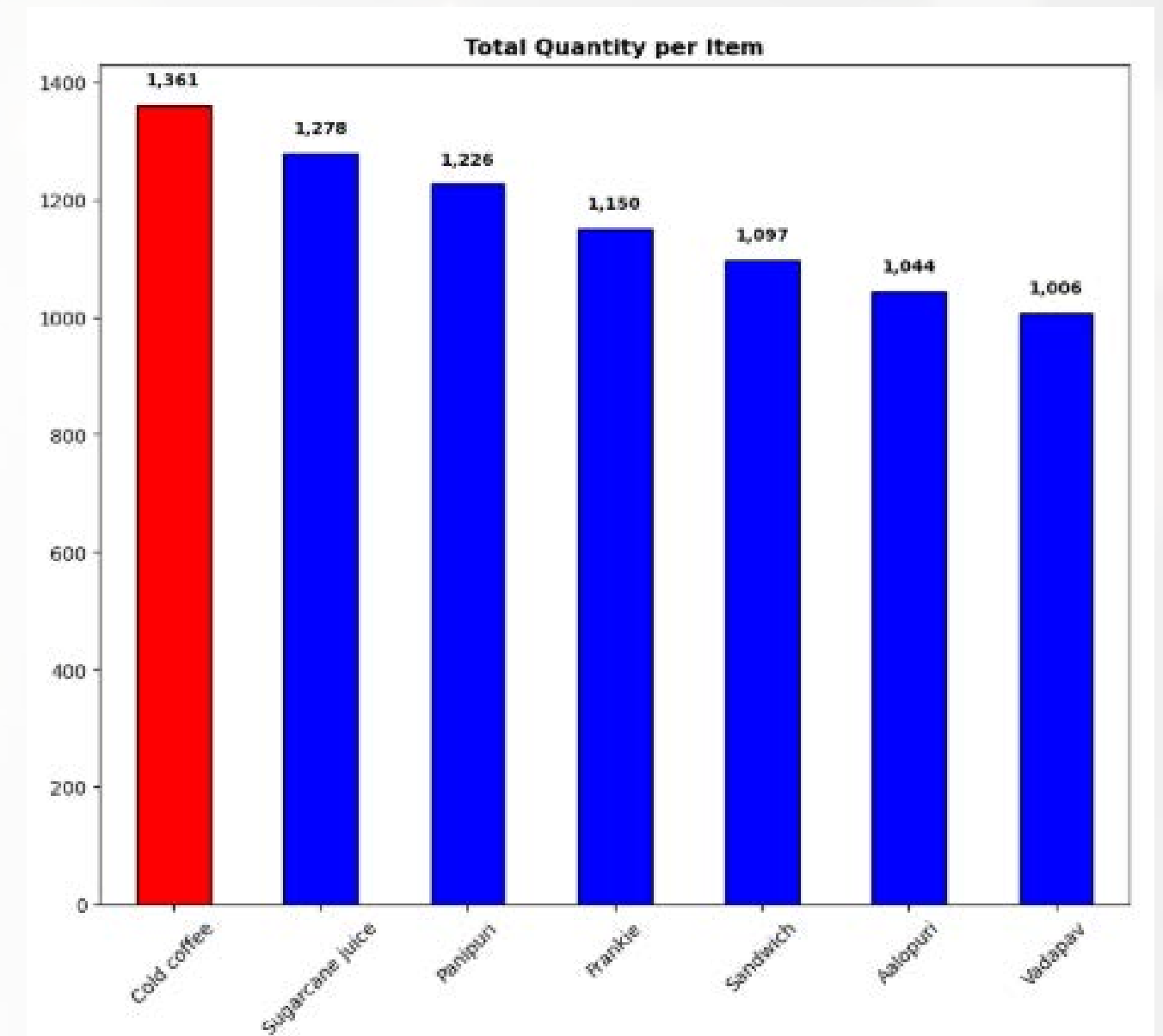
- karena $P\text{-value} < 0.05$, maka terdapat hubungan signifikan antara quantity dan transaction amount.
- Dapat di indikasikan juga bahwasannya quantity merupakan salah satu atribut yang mempengaruhi transaction amount.

- Scatter plot diatas menunjukkan hubungan antara jumlah item yang dibeli dengan transaction amount.
- Garis putus-putus diatas merupakan garis tren regresi liner pada scatter.
- **Dari Hasil scatter plot diatas menunjukkan adanya hubungan positif antara quantity dan juga transaction amount.**

VISUALIZATION OUTPUT



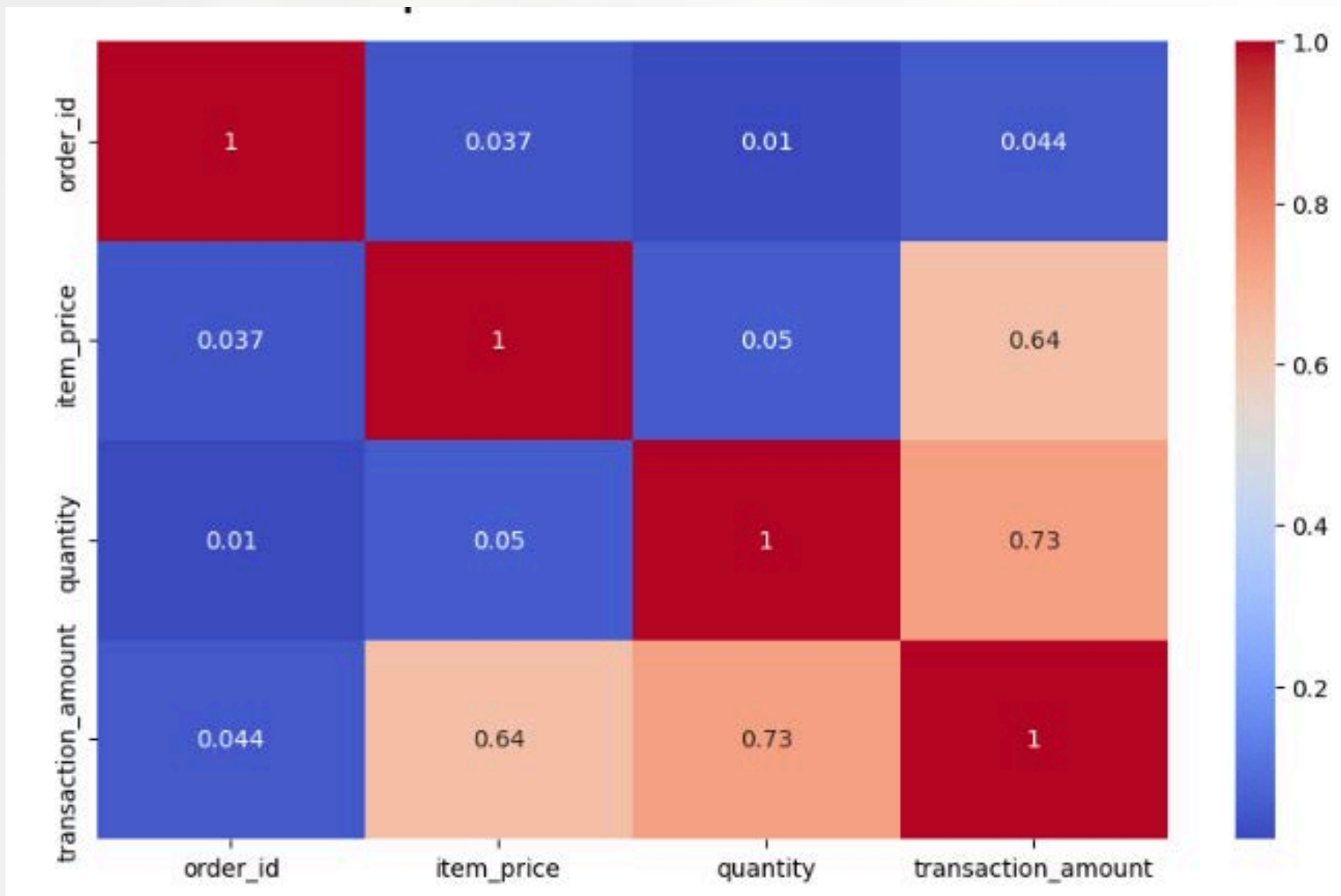
- Produk dengan transaction amount tertinggi adalah sandwich dengan total transaction amount sebesar 65,820



- Item dengan total quantity terbanyak adalah cold coffee yang mana dapat diindikasikan bahwasannya cold coffee merupakan produk dengan penjualan terbanyak.
- Dapat di indikasikan juga bahwasannya tidak selalu produk dengan penjualan terbanyak merupakan penyumbang terbesar ke transaction amount.

VISUALIZATION OUTPUT

Matrix Korelasi



#heatmap kemungkinan menggunakan uji pearson sehingga nilai korelasinya tidak sama dengan yang diuji sebelumnya

- **Korelasi transaction amount & order id:**

- Korelasi positif lemah (0.044) terlalu dekat dengan 0 menandakan hampir tidak ada hubungan antara order id dengan transaction amount karena order id berfungsi sebagai identifier transaksi dan bukan faktor yang mempengaruhi nilai penjualan.

- **Korelasi transaction amount & item price:**

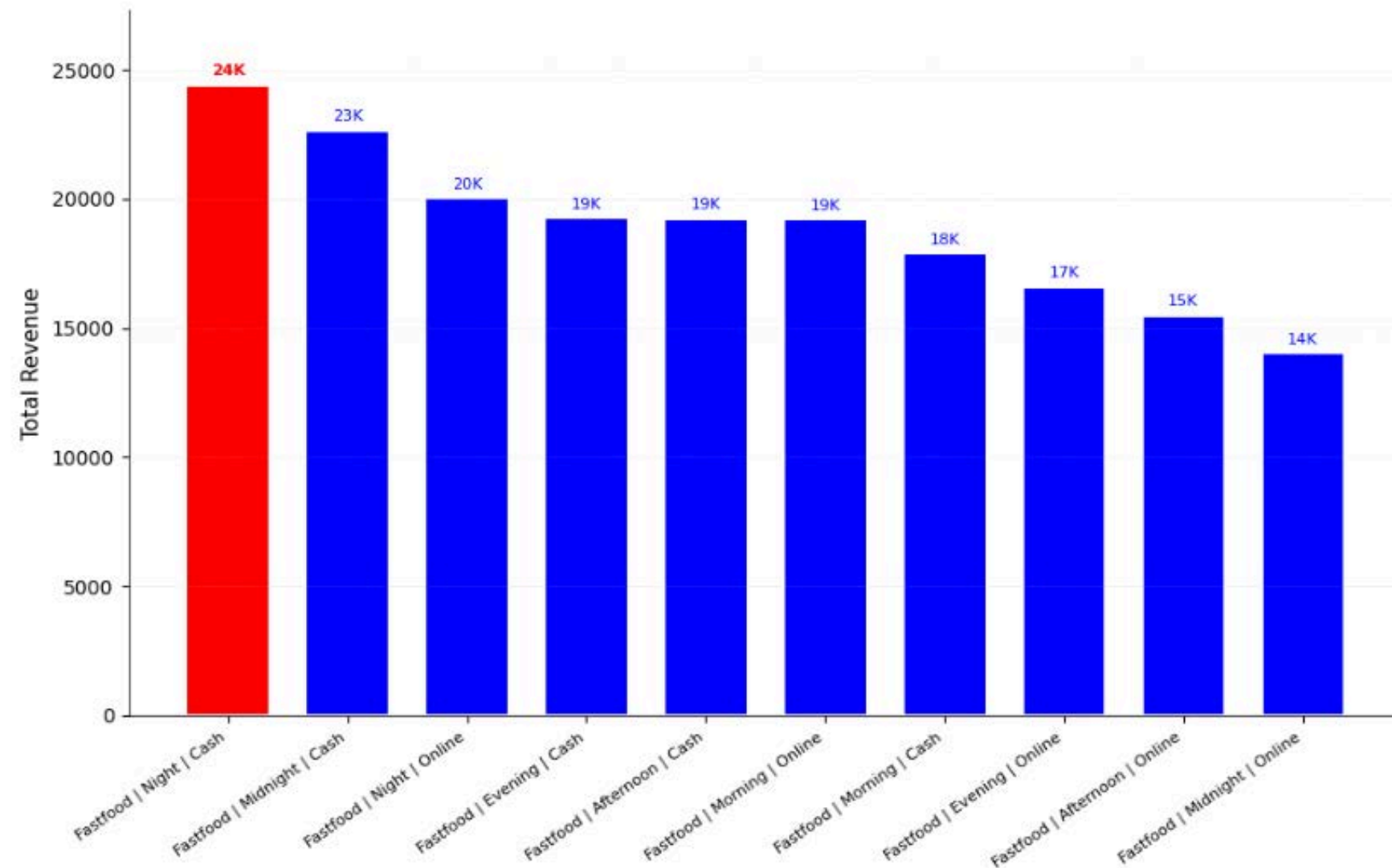
- Nilai korelasi sebesar 0.64 menunjukkan bahwa item dengan harga lebih tinggi cenderung menghasilkan transaction amount yang lebih besar. Hal ini mengindikasikan bahwa item price memiliki pengaruh yang cukup kuat terhadap nilai transaction amount pelanggan.

- **Korelasi transaction amount & quantity:**

- Korelasi sebesar 0.73 menunjukkan bahwa quantity item yang dibeli pelanggan memiliki hubungan yang kuat terhadap peningkatan nilai transaction amount. Semakin banyak quantity yang dibeli, maka transaction amount cenderung semakin tinggi.

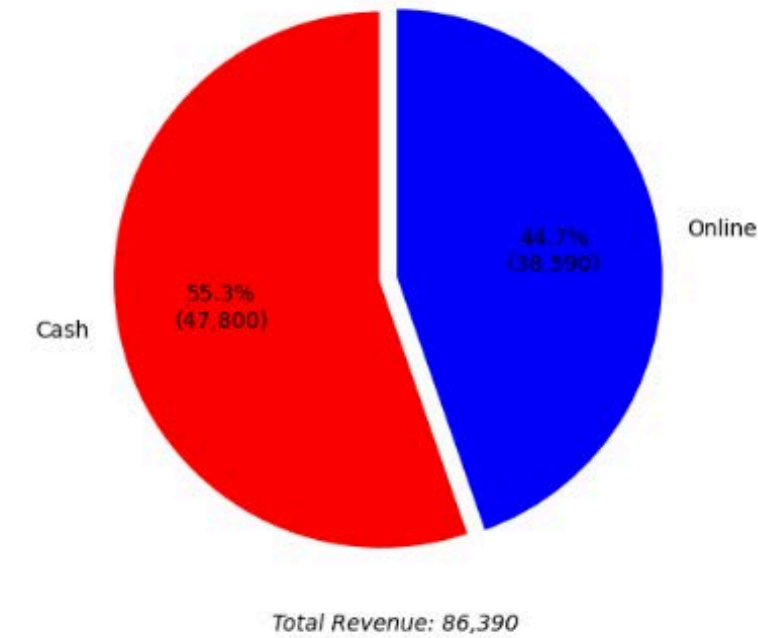
VISUALIZATION OUTPUT

Top 10 Kombinasi dengan Revenue Tertinggi
(Highlight: Bar Tertinggi Berwarna Merah)

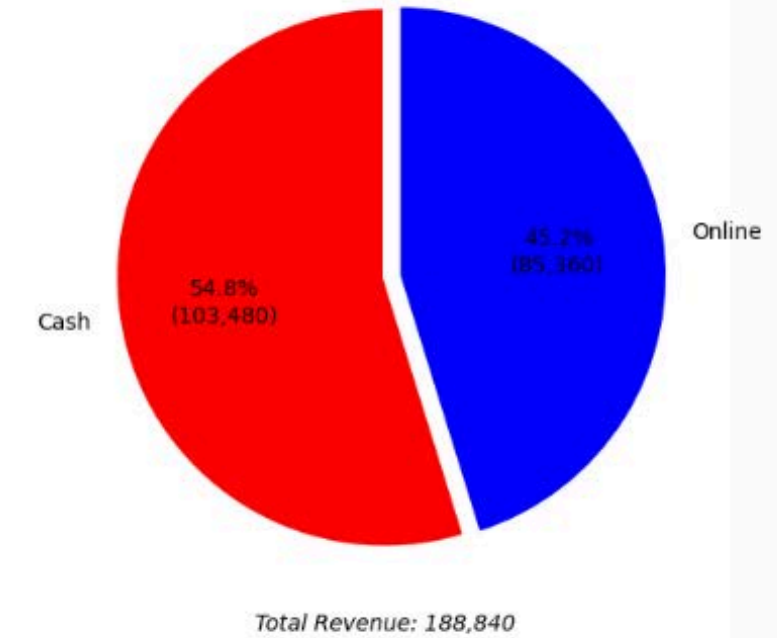


- Time of sale tertinggi dilihat dari jumlah pendapatannya:
 - Dapat dilihat pada bar-chart diatas bahwa penjualan di malam hari memiliki time of sale tertinggi jika dilihat dari transaction amountnya.

Beverages: Revenue proportion by Payment Method



Fastfood: Revenue proportion by Payment Method



- Dari pola pembayaran baik pada beverage dan fastfood relatif sama.
- Pembayaran cash masih mendominasi terhadap revenue.

INSIGHTS DAN RECOMENDATION

- **Berdasarkan analisis penjualan dilihat dari transaction amount dan quantity, terdapat beberapa item yang secara konsisten menghasilkan pendapatan tertinggi.** Seperti untuk jika dilihat dari transaction amount ada sandwich yang mendominasi dan pada quantity ada cold coffee. Kedua item tersebut dapat dijadikan sebagai produk signature dari balaji dan dijadikan tampilan awal menu.
- **Penjualan dengan revenue terbanyak ialah di malam hari.** Sehingga diperlukannya pengaturan jadwal staff yang lebih terfokus di malam hari, persediaan stock yang tepat, dan juga diadakannya promo night sale untuk menarik lebih banyak pembeli.
- **Metode pembayaran cash masih mendominasi total transaction.** Perlu disediakan stock uang untuk uang kembalian dan juga diskon untuk pembayaran online agar dapat menaikkan jumlah transaksi online agar proporsi antara online dan cash menjadi lebih seimbang.
- **Quantity dan item price memiliki korelasi yang kuat dengan transaction amount.** Terapkan promo minimum pembelian dan fokus pada item dengan high value.

PESONAL REFLECTION

- Memahami pentingnya proses data cleaning sebelum analisis dilakukan agar hasil analisis lebih akurat dan tidak bias.
- Realitas data dunia nyata pada proses persiapan data ini menegaskan bahwa dataset dunia nyata seringkali memiliki berbagai kendala kualitas. Dalam proyek ini, saya mengidentifikasi tantangan berupa nilai yang hilang (missing values), serta outliers pada transaction amountnya.
- Melatih kemampuan berpikir analitis dalam mengubah data mentah menjadi informasi yang dapat dipahami.
- Tahapan Feature Engineering belum tentu dapat dilakukan pada semua jenis persoalan salah satu contohnya pada dataset ini.
- Menyadari bahwa ketelitian sangat penting dalam proses analisis data karena kesalahan kecil dapat memengaruhi hasil akhir. Lebih teliti juga terhadap deadline projectnya (walaupun feedback tugas sebelumnya telat diberikan kepada students)
- Menyadari bahwasannya dengan adanya project ini membuka wawasan peneliti mengenai data secara nyata.

**THANK YOU
VERY MUCH**